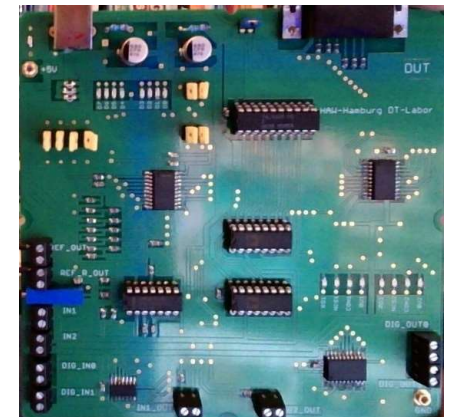
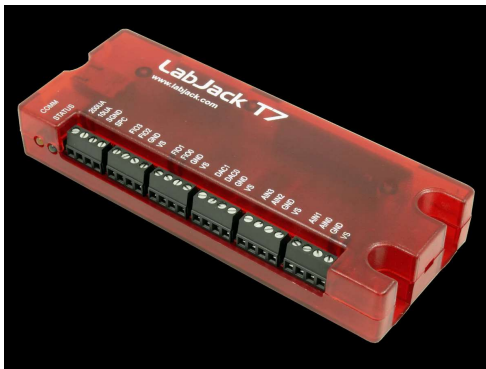


Embedded Systems Test & Verification

Lab session 2



Introduction

preliminary remark:

1.) completion of all tasks in one lab session is not expected!

More important is that tests are planned and performed carefully.

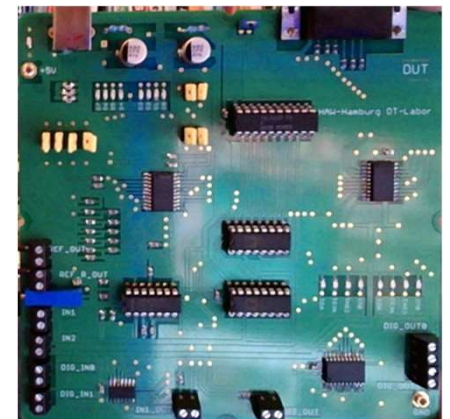
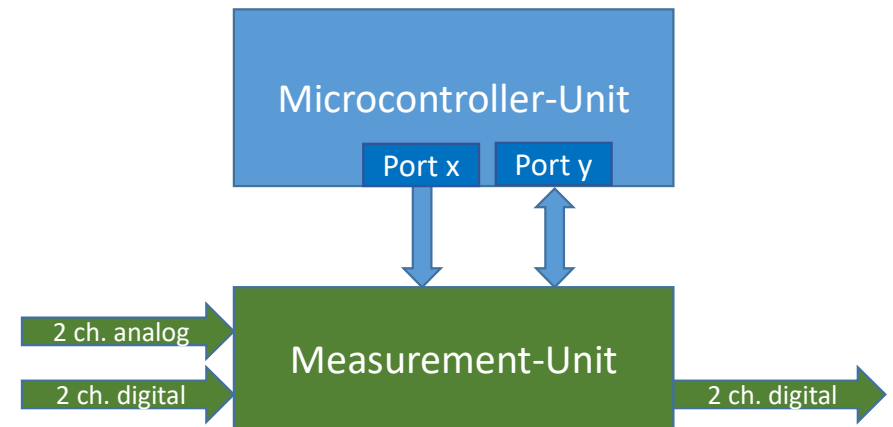
2.) Read this lab description carefully from start to end and after having read it completely decide on your option: DAC or ADC

Lab session 2 at a glance

- „the story“
 - the students take on the role of test engineers
 - the task is test-development
 - understand the device under test (DUT)
 - specify test cases
 - develop the test program
 - For test development a prototype version of the DUT is available
 - On this prototype some internal signals are accessible without need of a special test fixture (The final product would have test points for nails fixture instead).
 - The available test equipment
 - Test-PC (Lab PC or student's private PC)
 - LabJack T7-pro USB based measurement device

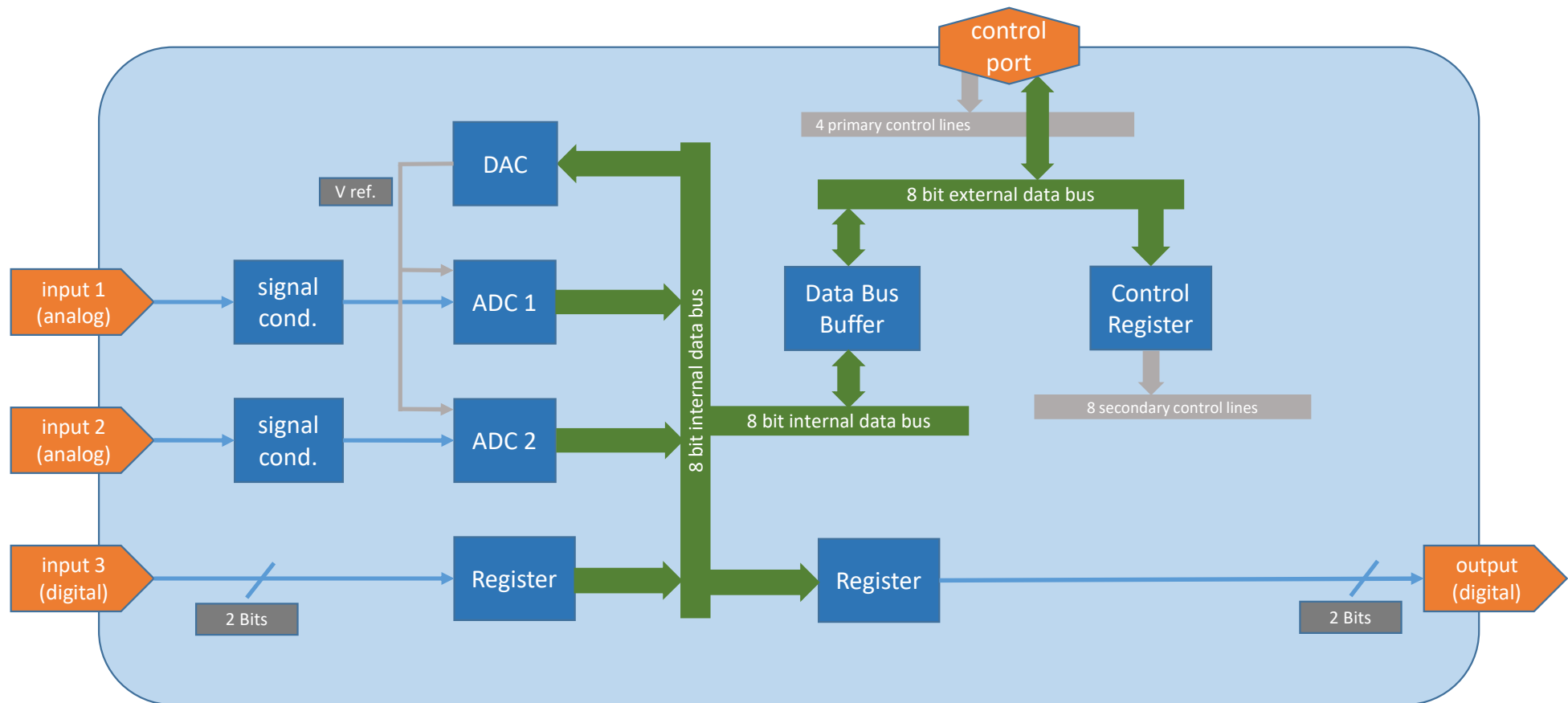
Device under Test (DUT)

- intended product: Measurement Unit
 - controllable via Microcontroller's GPIO (allows connection to Microcontrollers without external bus interface)
- Specification
 - 2 channel ADC, programmable input range
 - 2 digital inputs
 - 2 digital outputs
- For Test development
 - Only prototype board available, final product is not yet ready
- The measurement unit is NOT „state of the art“ (main components are from the „last century“)
 - intended for test training!



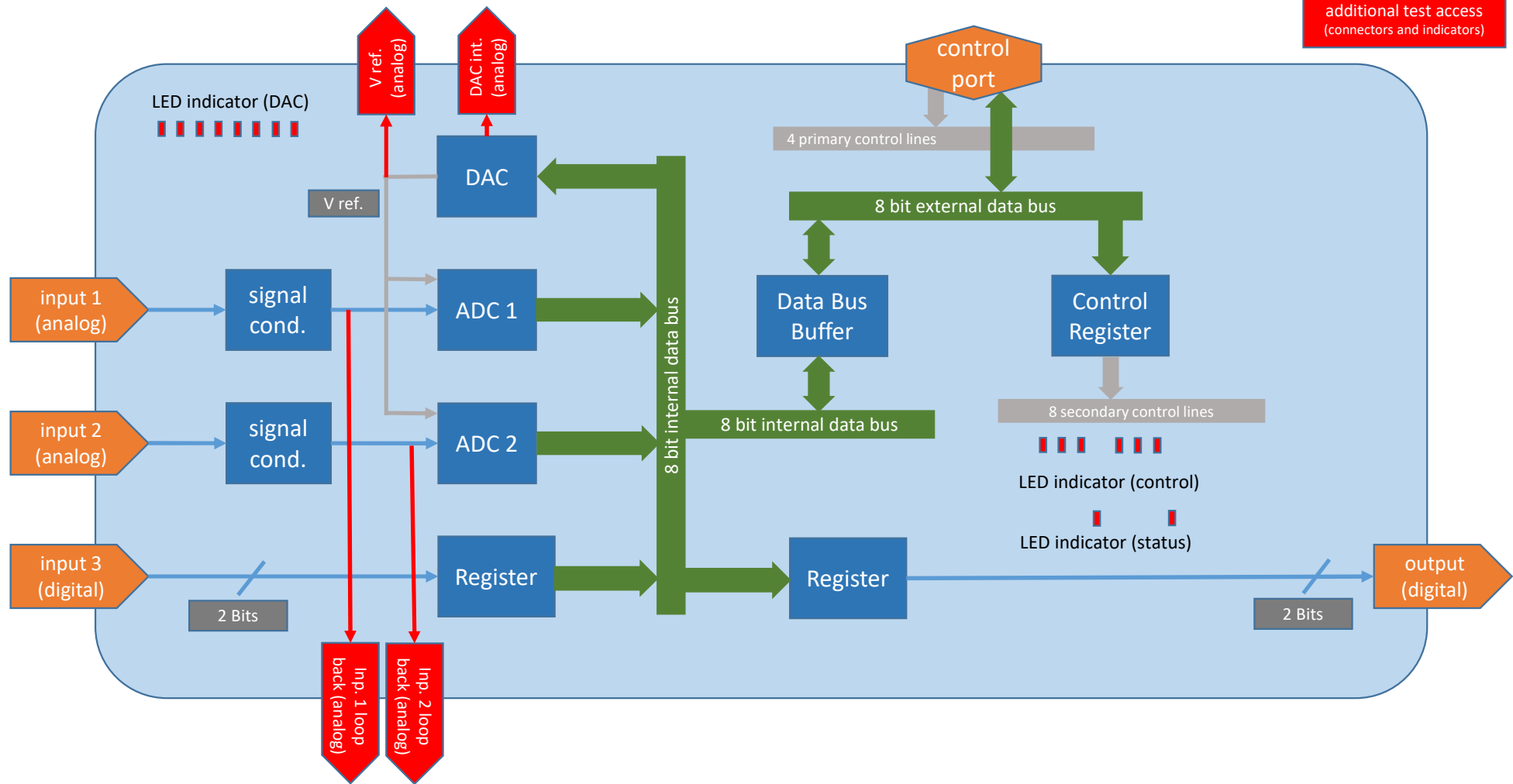
DUT block diagram (data-flow oriented)

operational I/Os

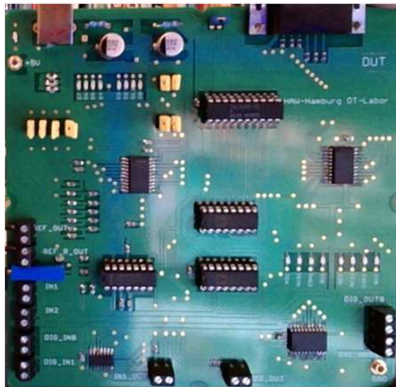


programmable input range via DAC setting

DUT prototype block diagram (incl. test access)

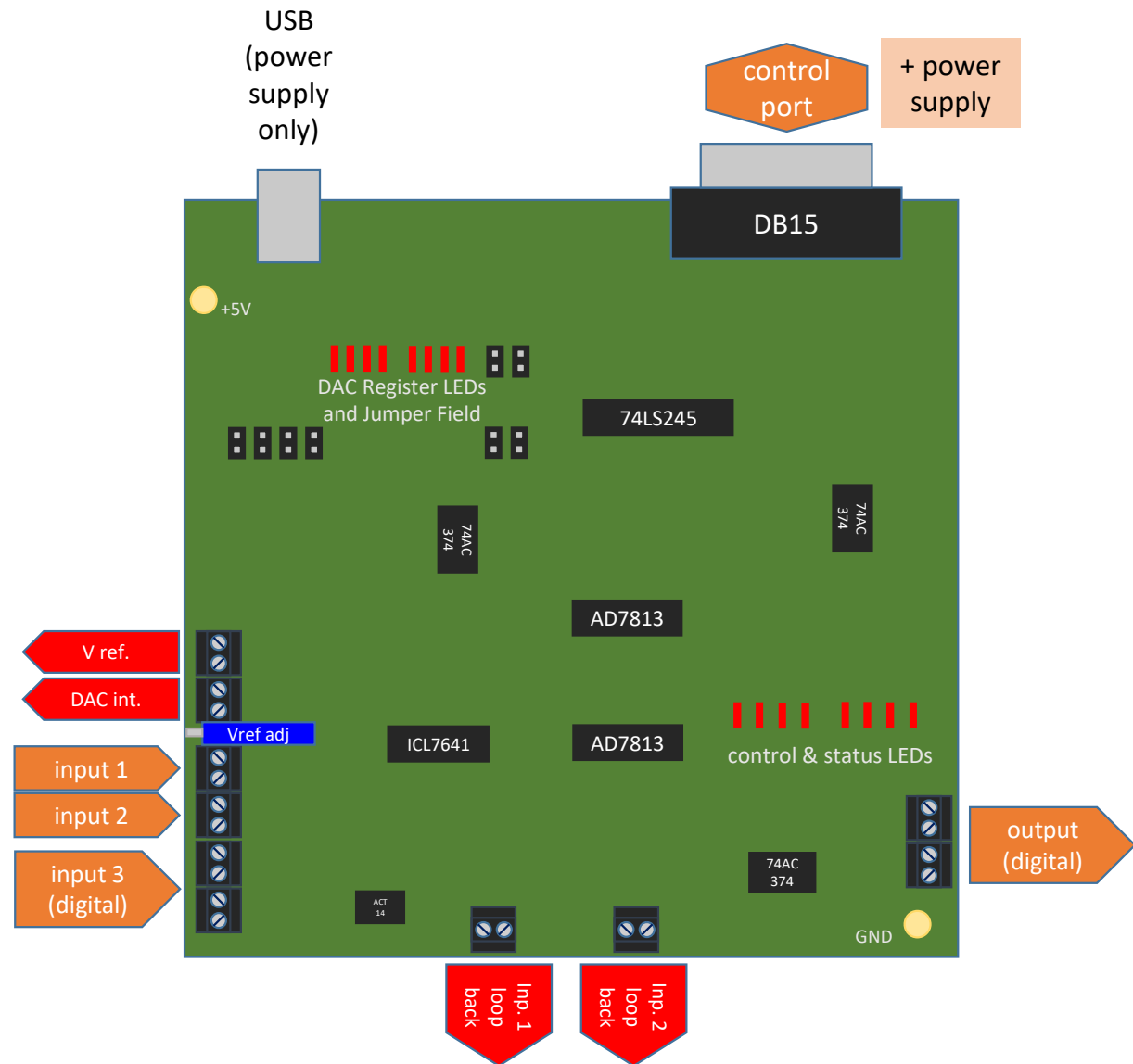


DUT prototype



USB: power supply connection
not necessary for lab

DB15: connection to LabJack extension port
enough power and sufficient GND from here!
screw terminals: always paired with one
terminal to GND, GND connection normally not
necessary if DB15 is connected



general preparation

- research and read data sheets for key components
 - ADC 7813 from Analog Devices
 - ICL 7641 from Maxim
 - 74LS245
 - 74AC374
 - ADC and DAC characteristics in general

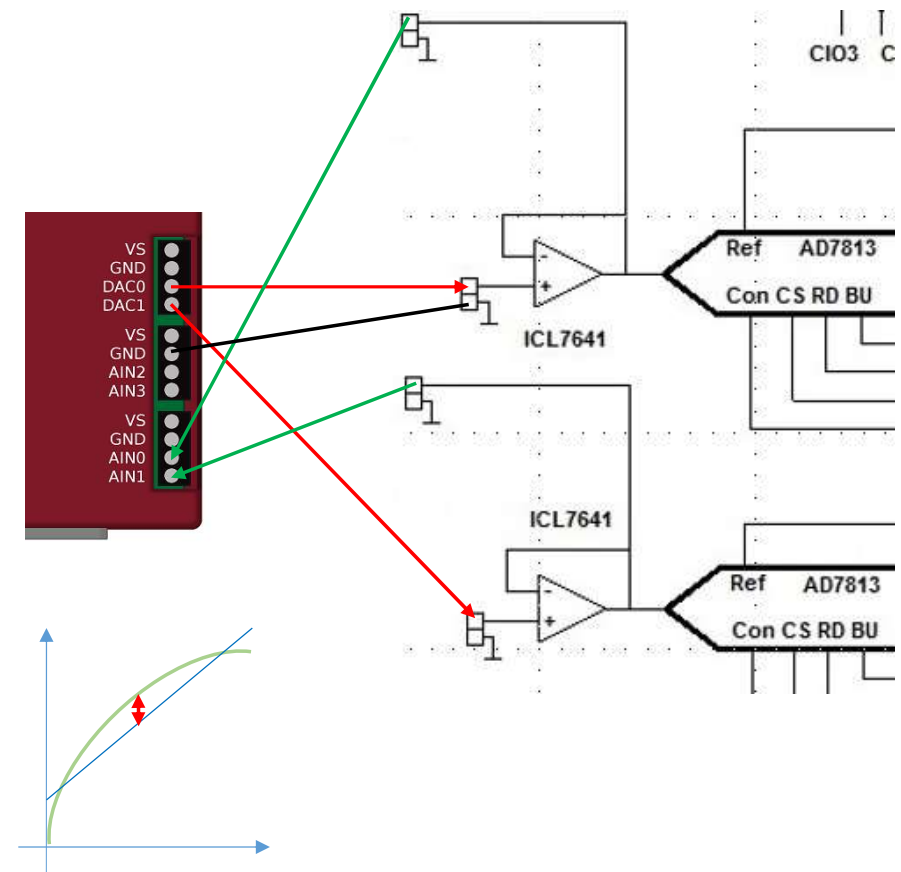
Task overview

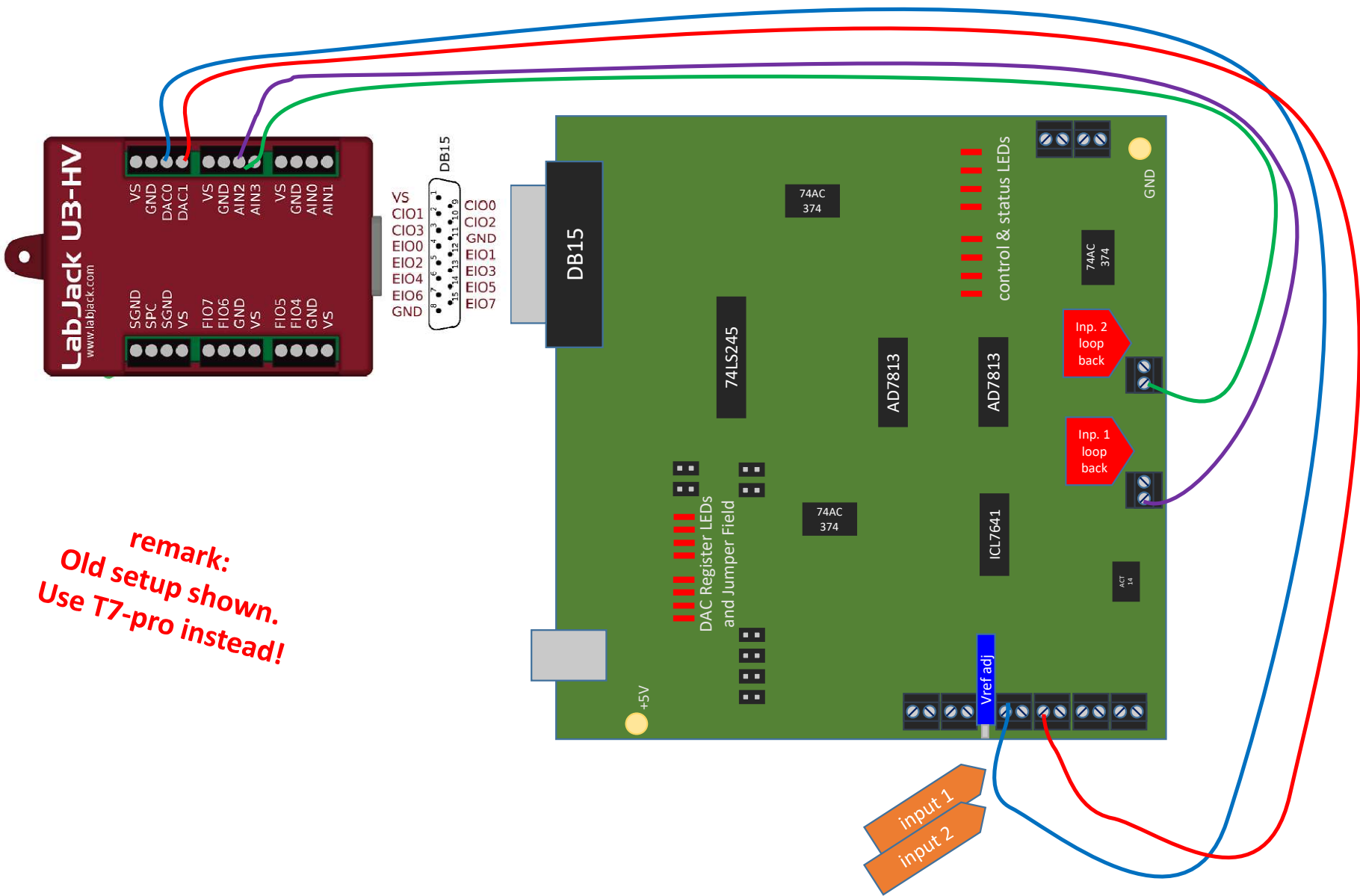
*preliminary remark:
completion of all tasks in one lab
session is not expected!
More important is that tests are
planned and performed carefully.*

- mandatory** • Test signal conditioning circuit
 - determine characteristic
 - gain, offset, linearity
- choose this** • Test DAC
 - basic functional test
 - determine characteristic (code to voltage), with and without LED jumpers
 - gain, offset, linearity
 - DNL and INL (differential and integral linearity)
- or this** • Test ADC
 - basic functional test
 - determine characteristic (voltage to code), for each ADC
 - 8-bit mode (10-bit mode only optional)
- optional
extra task** • Digital I/O
 - functional test

Task – Test signal conditioning circuit

- Questions
 - What kind of OPAMP circuit is implemented here?
 - What would be the ideal characteristic of this circuit?
- Test task in Python
 - For each channel
 - loop over full range with reasonable step size
 - apply stimulus voltage
 - measure response
 - store stimuli-response into an array
 - end loop
 - calculate
 - linear approximation
 - gain and offset
 - integral error (max. nonlinear deviation from linear approximation)
 - graphical output of characteristic
 - use Python package Matplotlib or external tool of your choice





remark:
Old setup shown.
Use T7-pro instead!

Preliminary remarks for the further tasks

- Control Access to DUT (via DB15 connector)
 - limited to 12 GPIO lines (intended microcontroller AND LabJack)
 - no bus interface means „bit-banging“ method must be used (find an explanation for bit-banging here: https://en.wikipedia.org/wiki/Bit_banging)
 - This also reflects what has to be taken into account for a pattern generator in a test system.
 - Plan EIO and CIO patterns carefully:
 - draw a timing diagram for each access needed
 - divide access timing into a sequence of actions and patterns
 - graphical in conjunction with the timing diagram
 - and as a **patterns** and **actions** list
 - consider if and when to change LabJack's EIO direction
 - consider if and when to change 245-type buffer's data direction
 - consider when to apply data
 - consider when to apply which control line
 - Define Python functions for elementary action sequences

Preliminary remarks - continued

- Bit-ordering of internal data bus compared with external data bus (EIO Bit-order) is reversed.
- This makes accesses a little bit tricky if EIO is used in byte-mode (which should be the normal way of access)
- Workaround (found somewhere on the internet; this method has probably fastest run-time):

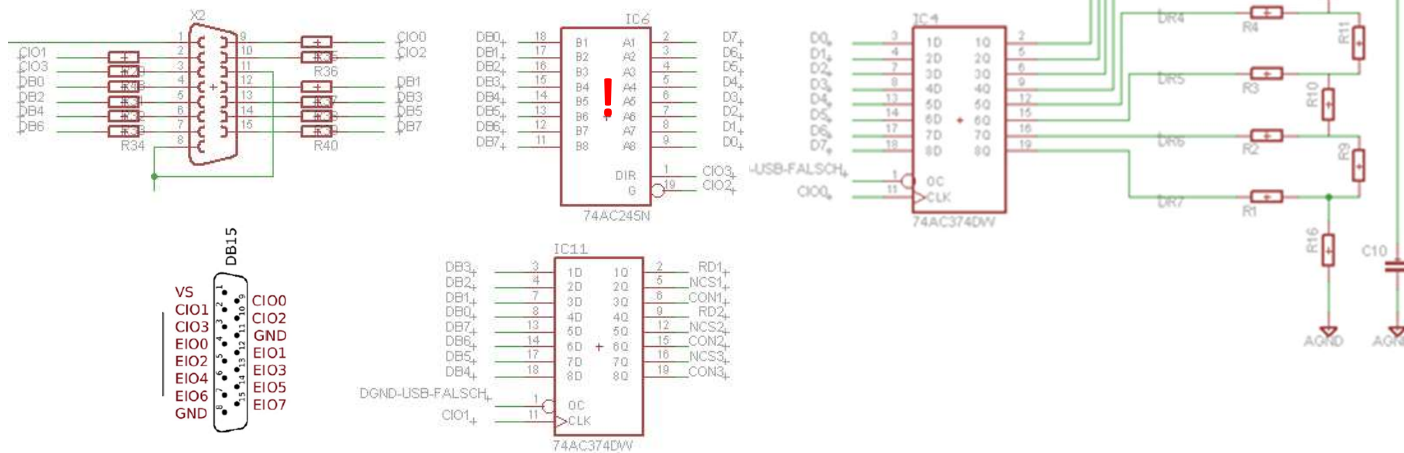
```
# reverse bit order Look-Up-Table
LUT = [0, 128, 64, 192, 32, 160, 96, 224, 16, 144, 80, 208, 48, 176, 112, 240,
      8, 136, 72, 200, 40, 168, 104, 232, 24, 152, 88, 216, 56, 184, 120,
      248, 4, 132, 68, 196, 36, 164, 100, 228, 20, 148, 84, 212, 52, 180,
      116, 244, 12, 140, 76, 204, 44, 172, 108, 236, 28, 156, 92, 220, 60,
      188, 124, 252, 2, 130, 66, 194, 34, 162, 98, 226, 18, 146, 82, 210, 50,
      178, 114, 242, 10, 138, 74, 202, 42, 170, 106, 234, 26, 154, 90, 218,
      58, 186, 122, 250, 6, 134, 70, 198, 38, 166, 102, 230, 22, 150, 86, 214,
      54, 182, 118, 246, 14, 142, 78, 206, 46, 174, 110, 238, 30, 158, 94,
      222, 62, 190, 126, 254, 1, 129, 65, 193, 33, 161, 97, 225, 17, 145, 81,
      209, 49, 177, 113, 241, 9, 137, 73, 201, 41, 169, 105, 233, 25, 153, 89,
      217, 57, 185, 121, 249, 5, 133, 69, 197, 37, 165, 101, 229, 21, 149, 85,
      213, 53, 181, 117, 245, 13, 141, 77, 205, 45, 173, 109, 237, 29, 157,
      93, 221, 61, 189, 125, 253, 3, 131, 67, 195, 35, 163, 99, 227, 19, 147,
      83, 211, 51, 179, 115, 243, 11, 139, 75, 203, 43, 171, 107, 235, 27,
      155, 91, 219, 59, 187, 123, 251, 7, 135, 71, 199, 39, 167, 103, 231, 23,
      151, 87, 215, 55, 183, 119, 247, 15, 143, 79, 207, 47, 175, 111, 239,
      31, 159, 95, 223, 63, 191, 127, 255]

# Bit order on DUT-internal Bus reversed with regard to EIO-Port
def reverseBitOrder(uint8):
    return LUT[uint8]
```

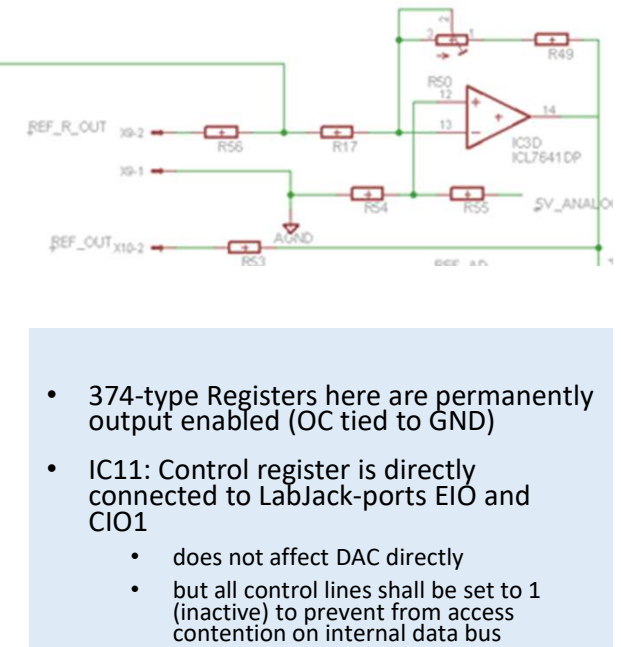
- **special hint:** As the DAC register is bit-reversing on it's own, using the reverseBitOrder function should not be used for the DAC!

Test DUT's DAC

DAC-related DUT circuits



- Prepare
 - R2R DAC in general
 - DAC characteristics
 - differential non-linearity
 - integral non-linearity
 - data-sheets for 74LS245 and 74AC374

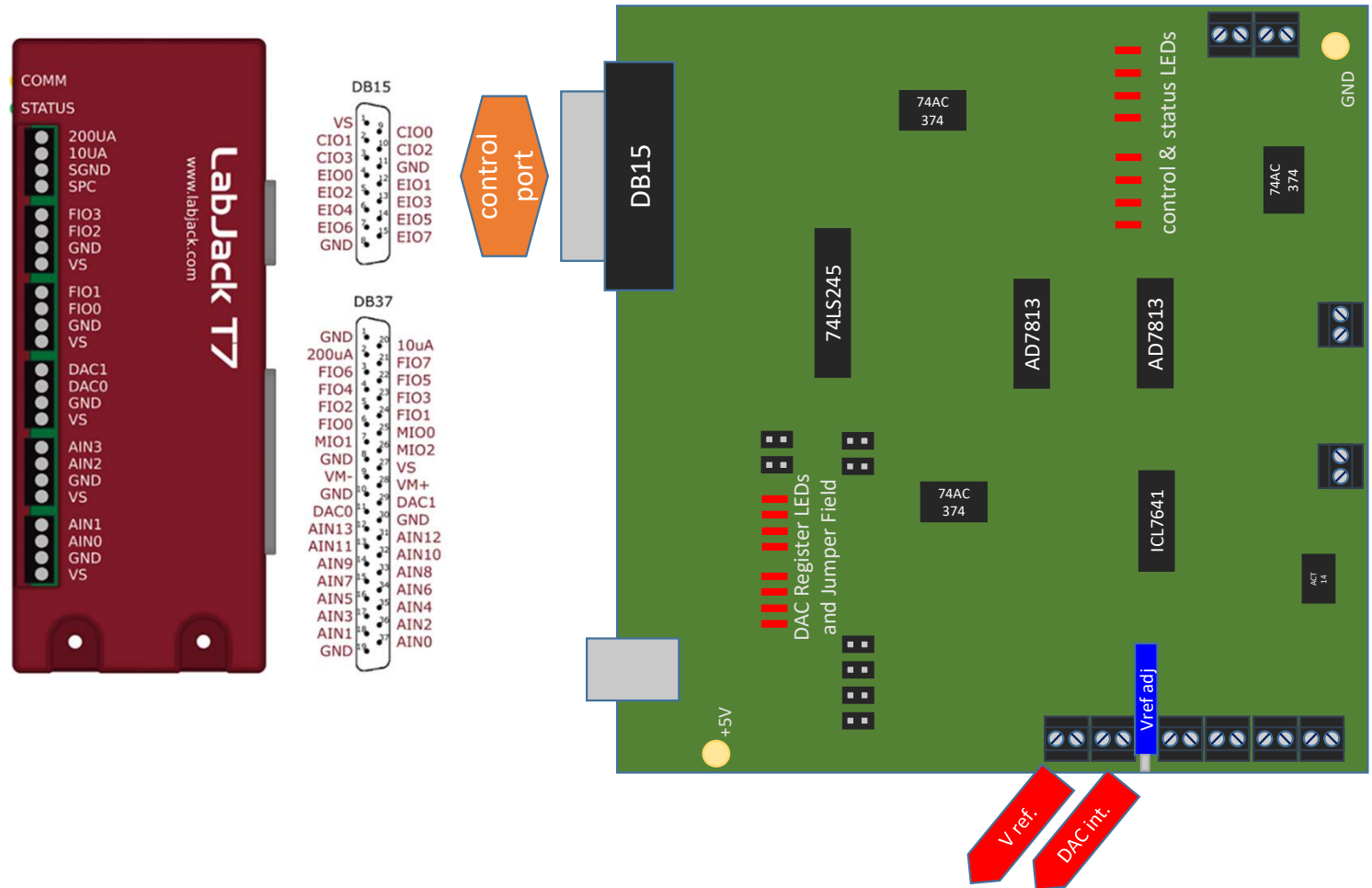


- 374-type Registers here are permanently output enabled (OC tied to GND)
- IC11: Control register is directly connected to LabJack-ports EIO and CIO1
 - does not affect DAC directly
 - but all control lines shall be set to 1 (inactive) to prevent from access contention on internal data bus
- IC6: bidirectional 245-type buffer
 - A-side on internal data bus
 - B-side connected to LabJack EIO
 - control lines connected to LabJack CIO2 and CIO3
 - enabling and direction switching must be considered carefully
- IC4: DAC data register
 - data input connected to internal data bus
 - control line (clock) connected to LabJack CIO0

Sub-Task: Test Plan

- Plan EIO and CIO patterns for DAC-access
 - draw a timing diagram for writing to the DAC-register
 - If you don't like drawing by hand, you can use this tool:
 - <https://wavedrom.com>
 - but it takes some training
 - divide access timing into a sequence of actions and patterns
 - graphical in conjunction with the timing diagram
 - and as a patterns and actions list
 - consider if and when to change JabJack EIO direction
 - consider if and when to change 245-type buffer's data direction
 - consider when to apply data
 - consider when to apply which control line
- Define Python functions for elementary action sequences

- control port
 - use it to set DAC register
- DAC internal / REF_R_OUT
 - direct output of R2R-network
- V ref. / REF_OUT
 - inverted version of DAC output
 - used as reference Voltage for the on-board ADCs



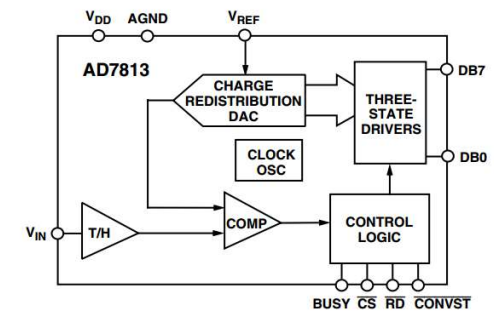
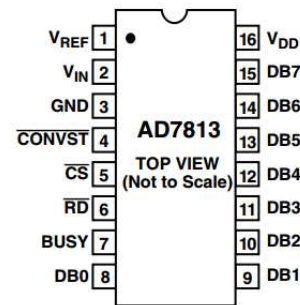
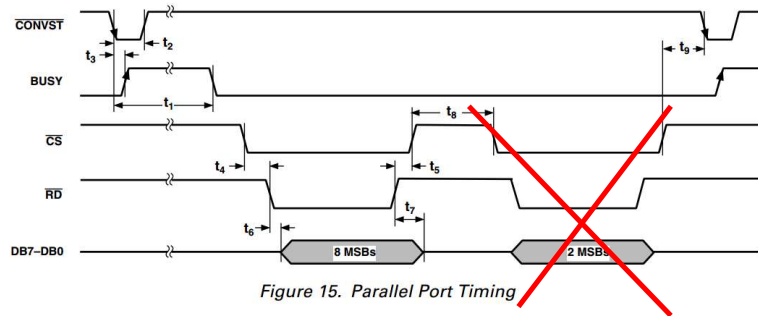
Sub-Task: Test Execution

- connect both, DAC and Reference to AIN channels
- Write a test task in Python:
- For each channel
 - loop for every DAC code
 - apply code to DAC
 - measure response
 - store code-response pair into an array
 - end loop
 - calculate
 - linear approximation
 - gain and offset
 - differential non-linear error
 - for each code-transition
 - maximum value
 - integral error
 - for each code
 - maximum value
 - graphical output of characteristics
 - use Python package Matplotlib or external tool of your choice

Test DUT's ADC

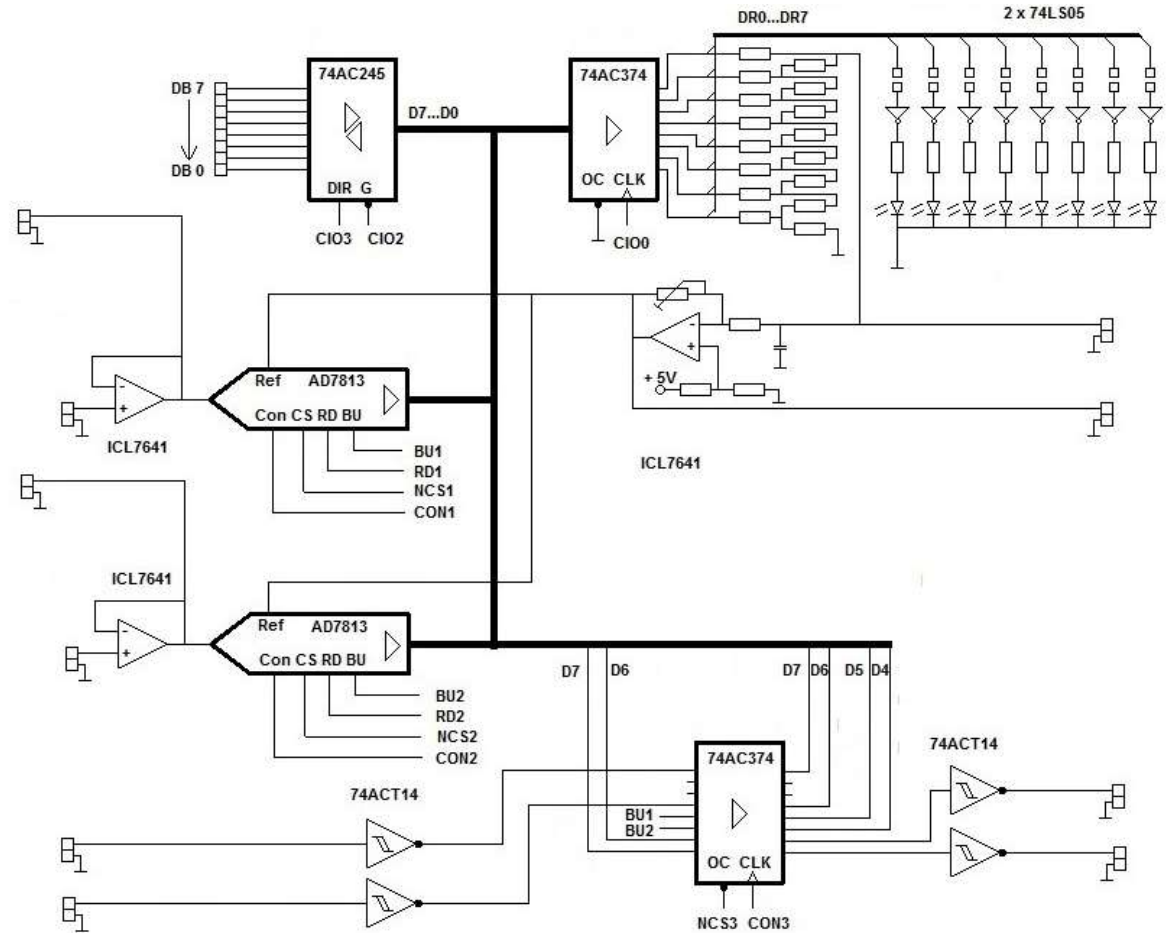
ADC

- Some remarks
 - Set reference voltage via DAC, kept constant throughout test execution.
 - Apply stimuli voltages to the DUT's analog inputs and read out ADCs' results.
 - As a first approach access is limited to 8 bit data transfer (10 bit optionally).



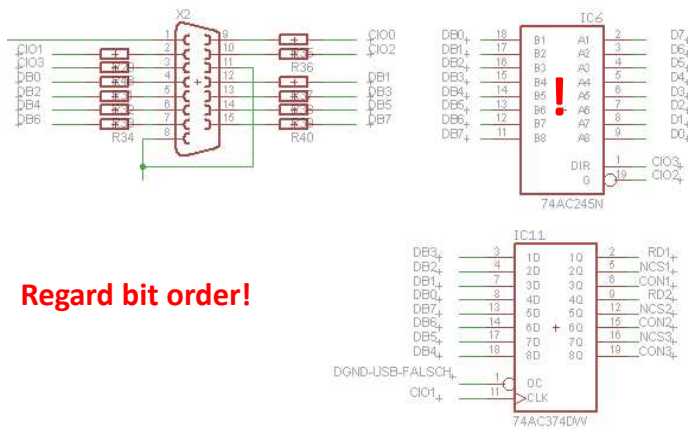
ADC-related DUT circuit

- less detailed partial circuit diagram
 - control register not shown
- The ADC Busy signal is available through the digital port's data register but practically not observable
 - too fast for test sequences

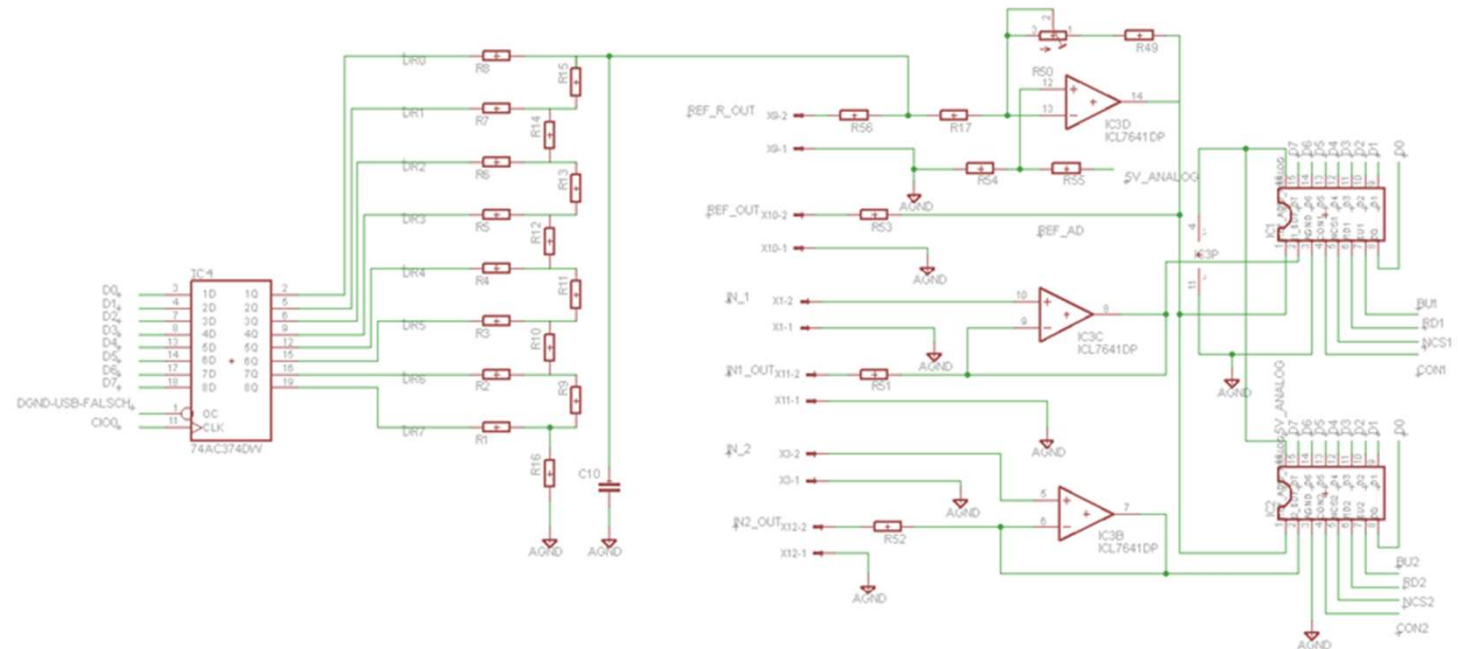


ADC-related DUT circuit

- more detailed snippets from circuit diagram



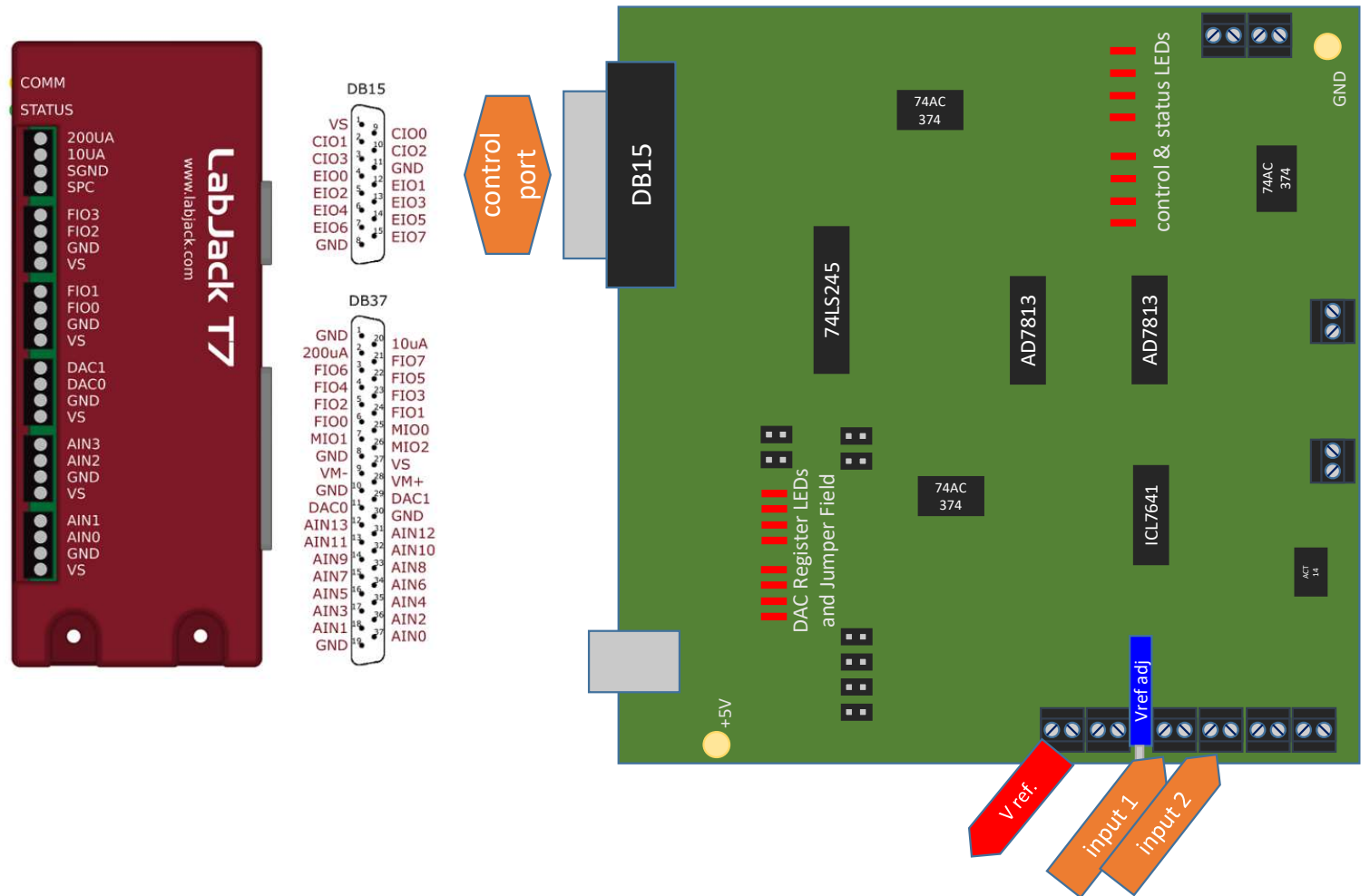
Regard bit order!



Sub-Task: Test Plan

- Plan EIO and CIO patterns for ADC-access
 - draw a timing diagram for reading from ADC (8-bit mode)
 - divide access timing into a sequence of actions and patterns
 - graphical in conjunction with the timing diagram
 - and as a patterns and actions list
 - consider if and when to change JabJack EIO direction
 - consider if and when to change 245-type buffer's data direction
 - consider when to read data
 - consider when to apply which control line
- Define Python functions for elementary action sequences
- Calculate the digital value for the DUT DAC to set reference Voltage to 3.5 V

- inputs 1 and 2
 - stimuli voltages from LabJack DAC 0 and 1
- V ref. / REF_OUT
 - observe reference Voltage for the on-board ADCs
 - to LabJack AIN
- Control Port
 - use it for setting of Vref via DAC
 - use it for reading of ADC values



Sub-Task: Test Execution

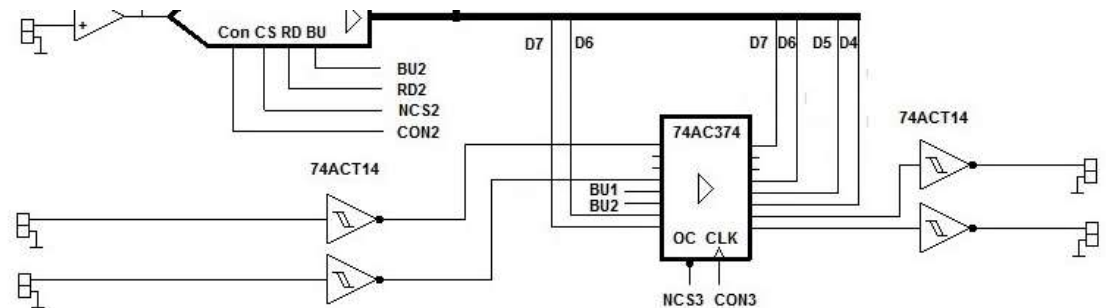
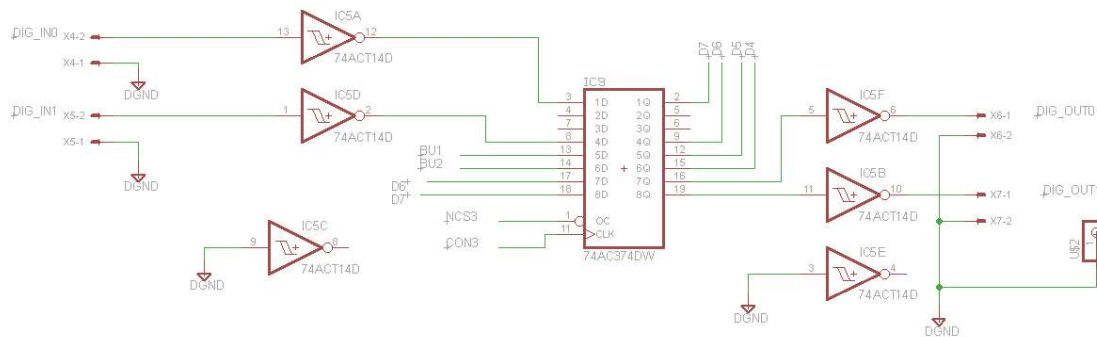
- connect DUT reference voltage to LabJack AIN channel
- connect LabJack DAC outputs to DUT input channels
- Write a test task in Python:
- Set reference Voltage to 3.5 V
 - observe V_{ref} and change digital input for DUT DAC if necessary
- For each input channel
 - loop from 0 V to 5 V with minimum possible stepsize (goes beyond valid range)
 - apply voltage to DUT input channel via LabJack DAC
 - force ADC to sample input voltage
 - read ADC data response via control port
 - store voltage-data pair into an array
 - end loop
 - calculate
 - linear approximation (only for valid range)
 - gain and offset (only for valid range)
 - differential non-linear error
 - for each code-transition
 - maximum value
 - integral error
 - for each code
 - maximum value
 - graphical output of characteristics
 - use Python package Matplotlib or external tool of your choice

Additional Task

Test Digital I/O

Test digital I/O

- digital port, digital strobe CON3 for write and read



Tasks

- Develop a test plan
 - Timing diagram, pattern sequence etc.
- Execute test
 - make the right LabJack-DUT connections
 - develop and perform a Python script
 - find a useful way of visualization